

Einführung in Data Science

Dimensionsreduktion



Eindimensionale EDA und Visualisierungen

Inhalte

1. Überblick & Grundbegriffe
2. Einführung in Python (NumPy und Pandas)
3. Eindimensionale EDA und Visualisierungen
4. Visualisierungen und mehrdimensionale EDA
5. Mehrdimensionale EDA
6. Dimensionsreduktion: PCA
- ➡ 7. Dimensionsreduktion: MDS, Isomap
8. Clustering: Algorithmen
9. Clustering: Validierung
10. Probeklausur
11. Feature Engineering, datengetriebene Modelle
12. Datengetriebene Modelle
13. Zeitreihen

Überblick &
Grundlagen

Explorative
(Daten-)
Analyse (EDA)

Feature
Engineering &
Modellierung

Heute

1. Wiederholung
2. Nachtrag PCA
3. Multidimensional Scaling & Isomap

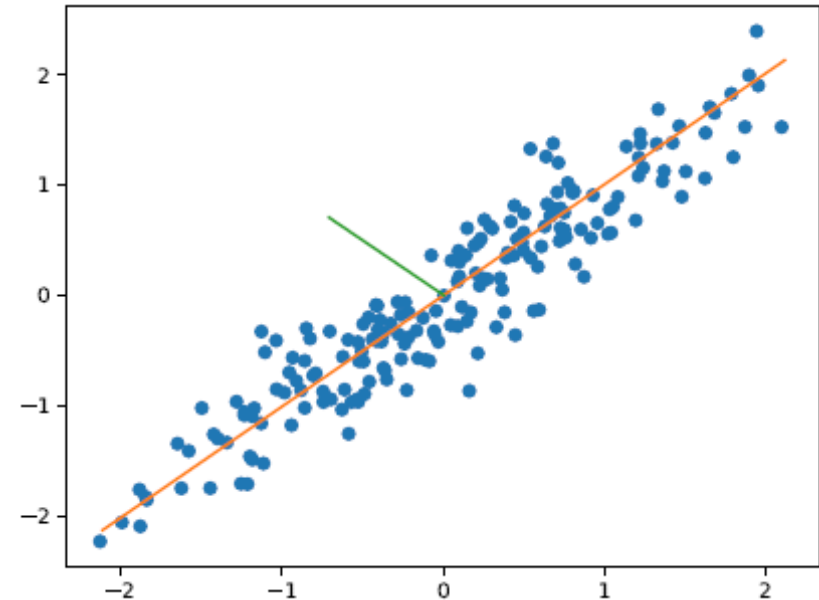
1. Wiederholung

1. Wiederholung

Hauptkomponentenanalyse

- Hauptkomponenten maximieren Varianz
- Hauptkomponenten sind orthogonal bzgl. vorheriger Hauptkomponenten
- Die „meisten“ Informationen stecken in den ersten Hauptkomponenten, d.h. für $d \ll p$ gilt

$$X_j = \sum_{i=1}^p \langle X_j, w_i \rangle w_i \approx \sum_{i=1}^d \langle X_j, w_i \rangle w_i$$



1. Wiederholung

Hauptkomponentenanalyse

- Für die 1. Hauptkomponente gilt

$$w_1 = \arg \max_{\|w\|=1} \sum_{i=1}^n \langle X_i, w \rangle^2.$$

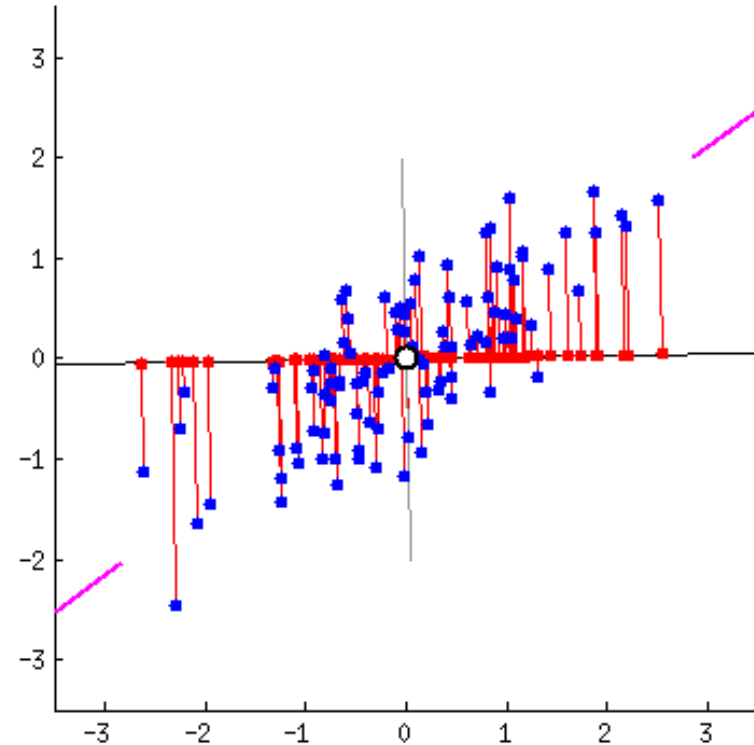
- Sei $\mathbf{X} \in \mathbb{R}^{n \times p}$ die Matrix mit den Beobachtungen $X_1, \dots, X_n$ als Zeilen. Dann gilt

$$w_1 = \arg \max_{\|w\|=1} \|\mathbf{X}w\|^2 = \arg \max_{\|w\|=1} w^T \mathbf{X}^T \mathbf{X} w.$$

- Das Maximum

$$\max_{\|w\|=1} \|\mathbf{X}w\|^2 = \max_{\|w\|=1} w^T \mathbf{X}^T \mathbf{X} w$$

ist die erklärte Varianz



Quelle: <https://stats.stackexchange.com/a/140579>

1. Wiederholung

Hauptkomponentenanalyse

- Wie können wir Hauptkomponenten effizient berechnen?
- Sei $\mathbf{X} \in \mathbb{R}^{n \times p}$ die Matrix mit den Beobachtungen $X_1, \dots, X_n$ als Zeilen
- Die Hauptkomponenten von $X_1, \dots, X_n$ entsprechen den Eigenvektoren von $\mathbf{X}^T \mathbf{X}$
- Die Eigenwerte von $\mathbf{X}$ entsprechen der erklärten Varianz

$$\lambda_k = \max_{\|w\|=1} \|\mathbf{X}w\|^2 \text{ und } v_k = \arg \max_{\|w\|=1} \|\mathbf{X}w\|^2$$

- Normalerweise: $p \ll n$ und $\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$ klein  Viele Daten, weniger Dimensionen
- Aber: manchmal sind die Daten hochdimensional, d.h. $n \ll p$
- Wie berechnen wir Hauptkomponenten effizient, falls $\mathbf{X}^T \mathbf{X}$ groß?

2. Nachtrag PCA

2. Nachtrag PCA

Hauptkomponentenanalyse

- Sei $\mathbf{X} \in \mathbb{R}^{n \times p}$ die Matrix mit den Beobachtungen $X_1, \dots, X_n$ als Zeilen
- Falls $p \gg n$, ist $\mathbf{X}^T \mathbf{X}$ groß, aber $\mathbf{X} \mathbf{X}^T$ klein
- Gibt es einen Zusammenhang zwischen Eigenvektoren und -werten von $\mathbf{X}^T \mathbf{X}$ und $\mathbf{X} \mathbf{X}^T$?
- Seien λ_i und v_i die Eigenwerte und -vektoren von $\mathbf{X}^T \mathbf{X}$, d.h.

$$\mathbf{X}^T \mathbf{X} v_i = \lambda_i v_i$$

$$\Leftrightarrow \mathbf{X} \mathbf{X}^T (\mathbf{X} v_i) = \lambda_i (\mathbf{X} v_i)$$

$$\Leftrightarrow \mathbf{X} \mathbf{X}^T u_i = \lambda_i u_i$$

$$\Leftrightarrow \mathbf{X}^T \mathbf{X} (\mathbf{X}^T u_i) = \lambda_i (\mathbf{X}^T u_i)$$

Wir können die Eigenwerte und -vektoren der kleineren Matrix bestimmen!

2. Nachtrag PCA

Hauptkomponentenanalyse

- Sei $\mathbf{X} \in \mathbb{R}^{n \times p}$ die Matrix mit den Beobachtungen $X_1, \dots, X_n$ als Zeilen
- $\mathbf{X}^T \mathbf{X} v_i = \lambda_i v_i \Leftrightarrow \mathbf{X} \mathbf{X}^T u_i = \lambda_i u_i$
- Berechne Eigenwerte und -vektoren der kleineren Matrix
 - Falls $p \ll n$: $\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$
 - Falls $p \gg n$: $\mathbf{X} \mathbf{X}^T \in \mathbb{R}^{n \times n}$
- Achtung: Eigenvektoren haben nicht automatisch die Länge 1, d.h. wir müssen normieren
- Beispiel: $p \gg n$
 1. Berechne λ_i, u_i sodass $\mathbf{X} \mathbf{X}^T u_i = \lambda_i u_i$
 2. Berechne $v_i = \mathbf{X}^T u_i$, dann gilt $\mathbf{X}^T \mathbf{X} v_i = \lambda_i v_i$
 3. Normiere $w_i = \frac{v_i}{\|v_i\|} = \frac{\mathbf{X}^T u_i}{\|\mathbf{X}^T u_i\|}$

2. Nachtrag PCA

Hauptkomponentenanalyse

- Wie kann der Code an hoch-dimensionale Daten angepasst werden?

1. Berechne λ_i, u_i sodass $\mathbf{X}\mathbf{X}^T u_i = \lambda_i u_i$
2. Berechne $v_i = \mathbf{X}^T u_i$, dann gilt $\mathbf{X}^T \mathbf{X} v_i = \lambda_i v_i$
3. Normiere $w_i = \frac{v_i}{\|v_i\|} = \frac{\mathbf{X}^T u_i}{\|\mathbf{X}^T u_i\|}$

```
def pca(X, n_components=2):  
    # Step 1: Center the data by subtracting the mean of each feature  
    X_centered = X - np.mean(X, axis=0)  
  
    # Step 2: Compute the covariance matrix  
    XtX = np.dot(X_centered.T, X_centered)  
  
    # Step 3: Perform eigenvalue decomposition  
    eigenvalues, eigenvectors = np.linalg.eigh(XtX)  
  
    # Step 4: Sort the eigenvectors by eigenvalue in descending order  
    sorted_indices = np.argsort(eigenvalues)[::-1]  
    eigenvalues_sorted = eigenvalues[sorted_indices]  
    eigenvectors_sorted = eigenvectors[:, sorted_indices]  
  
    # Step 5: Select the top 'n_components' eigenvectors  
    eigenvectors_selected = eigenvectors_sorted[:, :n_components]  
  
    return eigenvectors_selected, eigenvalues_sorted[:n_components]
```

3. Multidimensional Scaling und Isomap

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

- Familie von Verfahren zur Visualisierung von Punkten und zur Dimensionsreduktion
- Wir betrachten das „klassische“ Verfahren
 - Torgerson-Scaling
 - Principal Coordinate Analysis (PCoA)
- Grundlage weiterer Verfahren, z.B. Isomap zur Extraktion nicht-linearer Mannigfaltigkeiten

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

Gegeben

- **Paarweise** Distanzen zwischen n Objekten
- Genauer: Matrix der paarweisen Distanzen $D = (d_{ij})_{i,j=1}^n$

Gesucht

- Datenmatrix **X**, mit Koordinaten für jedes der n Objekte, sodass die Objekte die paarweisen Distanzen aus D haben

Wichtig

- MDS startet mit Distanzmatrix D
- Datenmatrix **X** meist unbekannt
- PCA startet mit **X**

3. Multidimensional Scaling & Isomap

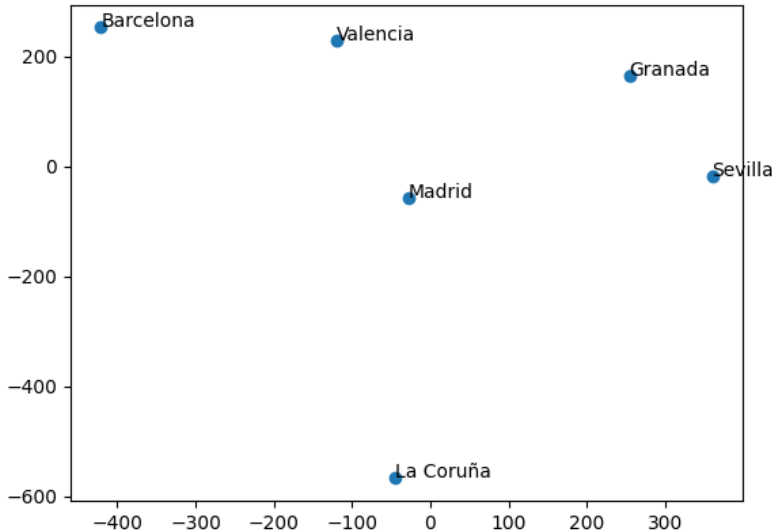
Code in `topic07_mds/mds_example.py`

Multidimensional Scaling (MDS)

Beispiel

- Gegeben: Distanzen
- Gesucht: Koordinaten

Ziel Start	Barcelona	Granada	La Coruña	Madrid	Sevilla	Valencia
Barcelona	0	680	900	500	830	300
Granada	680	0	790	360	210	380
La Coruña	900	790	0	510	680	800
Madrid	500	360	510	0	390	300
Sevilla	830	210	680	390	0	540
Valencia	300	380	800	300	540	0



Lösung nicht eindeutig!
Distanzen invariant
unter orthogonalen
Transformationen
(Rotation, Spiegelung)
und Translation



Quelle: Algorithmen, Zufall, Unsicherheit und Pizza!

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

Wie können wir mit MDS die Dimension reduzieren?

1. Sei $\mathbf{X} \in \mathbb{R}^{n \times p}$ die Matrix mit den Beobachtungen $X_1, \dots, X_n$ als Zeilen
 2. Bestimme die paarweisen Distanzen der Beobachtungen $D = (d_{ij})_{i,j=1}^n$
 3. Mit MDS: Bestimme aus D neue Datenmatrix $\tilde{\mathbf{X}}$ mit $d < p$ Dimensionen
- $\tilde{\mathbf{X}}$ so, dass Abstände möglichst nah an ursprünglichen Abständen d_{ij}

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

- Gegeben: Paarweise Distanzen zwischen n Objekten: $D = (d_{ij})_{i,j=1}^n$
- Gesucht: Datenmatrix $\mathbf{X}$ mit Abständen aus D

Algorithmus:

1. Quadriere Einträge der Matrix D , d.h. $\tilde{D}(d_{ij}^2)_{i,j=1}^n$
- ➡ 2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$
3. Konstruiere $\mathbf{X}$ durch Eigenwertzerlegung von B

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$

- $\mathbf{X}$ ist unbekannt, aber: Dimensionen bekannt $\mathbf{X} \in \mathbb{R}^{n \times p}$
- Für $B = (b_{ij})_{i,j=1}^p$ gilt

$$b_{ij} = \sum_{k=1}^p x_{ik} x_{jk} \quad \boxed{1}$$

- Für die quadrierten Abstände gilt

$$d_{ij}^2 = \sum_{k=1}^p (x_{ik} - x_{jk})^2 = \sum_{k=1}^p x_{ik}^2 - 2x_{ik}x_{jk} + x_{jk}^2 = b_{ii} - 2b_{ij} + b_{jj} \quad \boxed{2}$$

- Der Mittelwertsvektor der Daten sei 0, d.h.

$$\sum_{i=1}^n x_{ij} = 0 \quad \forall j = 1, \dots, p \quad \boxed{3}$$

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$

➤ Aus (1) und (3) folgt

$$\sum_{i=1}^n b_{ij} = \sum_{i=1}^n \sum_{k=1}^p x_{ik} x_{jk} = 0 \quad \boxed{4}$$

$$\sum_{j=1}^n b_{ij} = \sum_{j=1}^n \sum_{k=1}^p x_{ik} x_{jk} = 0 \quad \boxed{5}$$

$$\sum_{i,j=1}^n b_{ij} = \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^p x_{ik} x_{jk} = 0 \quad \boxed{6}$$

$$\boxed{1} \quad b_{ij} = \sum_{k=1}^p x_{ik} x_{jk}$$

$$\boxed{2} \quad d_{ij}^2 = b_{ii} - 2b_{ij} + b_{jj}$$

$$\boxed{3} \quad \sum_{i=1}^n x_{ij} = 0 \quad \forall j = 1, \dots, p$$

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$

➤ Aus (2) und (4) folgt

$$\sum_{i=1}^n d_{ij}^2 = \sum_{i=1}^n b_{ii} - 2 \sum_{i=1}^n b_{ij} + \sum_{i=1}^n b_{jj} = \text{tr}(B) + n \cdot b_{jj} \Leftrightarrow b_{jj} = \frac{1}{n} \left(\sum_{i=1}^n d_{ij}^2 - \text{tr}(B) \right) \quad \boxed{7}$$

wobei tr die Spur bezeichnet

➤ Aus (2) und (5) und (6) folgt analog

$$\sum_{j=1}^n d_{ij}^2 = \sum_{j=1}^n b_{ii} - 2 \sum_{j=1}^n b_{ij} + \sum_{j=1}^n b_{jj} = n \cdot b_{ii} + \text{tr}(B) \Leftrightarrow b_{ii} = \frac{1}{n} \left(\sum_{j=1}^n d_{ij}^2 - \text{tr}(B) \right) \quad \boxed{8}$$

$$\sum_{i,j=1}^n d_{ij}^2 = \sum_{i,j=1}^n b_{ii} - 2 \sum_{i,j=1}^n b_{ij} + \sum_{i,j=1}^n b_{jj} = 2n \cdot \text{tr}(B) \Leftrightarrow 2 \text{tr}(B) = \frac{1}{n} \sum_{i,j=1}^n d_{ij}^2 \quad \boxed{9}$$

$$\boxed{2} \quad d_{ij}^2 = b_{ii} - 2b_{ij} + b_{jj}$$

$$\boxed{4} \quad \sum_{i=1}^n b_{ij} = 0$$

$$\boxed{5} \quad \sum_{j=1}^n b_{ij} = 0$$

$$\boxed{6} \quad \sum_{i,j=1}^n b_{ij} = 0$$

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$

- Löse (2) nach b_{ij} auf: $b_{ij} = -\frac{1}{2}(d_{ij}^2 - b_{ii} - b_{jj})$
- Aus (7) bis (9) folgt

$$\begin{aligned} b_{ij} &= -\frac{1}{2} \left[d_{ij}^2 - \frac{1}{n} \left(\sum_{j=1}^n d_{ij}^2 - \text{tr}(B) \right) - \frac{1}{n} \left(\sum_{i=1}^n d_{ij}^2 - \text{tr}(B) \right) \right] \\ &= -\frac{1}{2} \left[d_{ij}^2 - \frac{1}{n} \sum_{j=1}^n d_{ij}^2 - \frac{1}{n} \sum_{i=1}^n d_{ij}^2 + \frac{1}{n} 2 \cdot \text{tr}(B) \right] \\ &= -\frac{1}{2} \left[d_{ij}^2 - \frac{1}{n} \sum_{j=1}^n d_{ij}^2 - \frac{1}{n} \sum_{i=1}^n d_{ij}^2 + \frac{1}{n^2} \sum_{i,j=1}^n d_{ij}^2 \right] \end{aligned}$$

$$\Rightarrow B = -\frac{1}{2} H \tilde{D} H$$

$$2 \quad d_{ij}^2 = b_{ii} - 2b_{ij} + b_{jj}$$

$$7 \quad b_{jj} = \frac{1}{n} \left(\sum_{i=1}^n d_{ij}^2 - \text{tr}(B) \right)$$

$$8 \quad b_{ii} = \frac{1}{n} \left(\sum_{j=1}^n d_{ij}^2 - \text{tr}(B) \right)$$

$$9 \quad 2 \text{tr}(B) = \frac{1}{n} \sum_{i,j=1}^n d_{ij}^2$$

$$H = I - \frac{1}{n} \mathbf{1} \cdot \mathbf{1}^T = \begin{pmatrix} 1 - \frac{1}{n} & -\frac{1}{n} & \cdots & -\frac{1}{n} \\ -\frac{1}{n} & 1 - \frac{1}{n} & \ddots & \vdots \\ \vdots & \ddots & \ddots & -\frac{1}{n} \\ 1 - \frac{1}{n} & 1 - \frac{1}{n} & -\frac{1}{n} & 1 - \frac{1}{n} \end{pmatrix}$$

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

- Gegeben: Paarweise Distanzen zwischen n Objekten: $D = (d_{ij})_{i,j=1}^n$
- Gesucht: Datenmatrix $\mathbf{X}$ mit Abständen aus D

Algorithmus:

1. Quadriere Einträge der Matrix D , d.h. $\tilde{D} = (d_{ij}^2)_{i,j=1}^n$ ✓
2. Gebe Matrix $B \in \mathbb{R}^{n \times n}$ als Funktion von $\tilde{D}$ an, wobei $B = \mathbf{X}\mathbf{X}^T$ ✓

$$B = -\frac{1}{2}H\tilde{D}H \text{ mit } H = I - \frac{1}{n}\mathbf{1} \cdot \mathbf{1}^T$$

- ➡ 3. Konstruiere $\mathbf{X}$ durch Eigenwertzerlegung von B

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

3. Konstruiere $\mathbf{X}$ durch Eigenwertzerlegung von B

- Wir kennen $\mathbf{X}$ nicht, aber wissen, dass $B = \mathbf{X}\mathbf{X}^T$
- D.h. B ist symmetrisch (und reellwertig)

$$B^T = (\mathbf{X}\mathbf{X}^T)^T = (\mathbf{X}^T)^T \mathbf{X}^T = \mathbf{X}\mathbf{X}^T = B$$

- Also gilt $B = V\Lambda V^T$
 - Da V orthogonal, gilt $V^{-1} = V^T$
 - Damit folgt $B = V\Lambda V^{-1} \Leftrightarrow BV = V\Lambda$
 - Insbesondere sind die Spalten von V die Eigenvektoren von B
- Wir wissen $\mathbf{X}\mathbf{X}^T = B = V\Lambda V^T$
- Also $\mathbf{X} = V\Lambda^{\frac{1}{2}}$
- Für $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ konstruieren wir $\mathbf{X} = V\Lambda^{\frac{1}{2}} = (\sqrt{\lambda_1}v_1, \dots, \sqrt{\lambda_p}v_p)$

Lineare Algebra

(Reelle) symmetrische Matrizen sind orthogonal diagonalisierbar:

- Sei B eine symmetrische Matrix mit Eigenwerten $\lambda_1, \dots, \lambda_n$
- Sei Λ die Diagonalmatrix der Eigenwerte
- Dann gibt es eine orthogonale Matrix V , sodass $B = V\Lambda V^T$ (spektrale Zerlegung)

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

- Gegeben: Paarweise Distanzen zwischen n Objekten: $D = (d_{ij})_{i,j=1}^n$
- Gesucht: Datenmatrix $\mathbf{X}$ mit Abständen aus D

Algorithmus:

1. Quadriere Einträge der Matrix D , d.h. $\tilde{D} = (d_{ij}^2)_{i,j=1}^n$

2. Berechne Matrix $B \in \mathbb{R}^{n \times n}$ für $H = I - \frac{1}{n} \mathbf{1} \cdot \mathbf{1}^T$ als

$$B = -\frac{1}{2} H \tilde{D} H$$

3. Konstruiere $\mathbf{X}$ durch Eigenwertzerlegung von B (Eigenwerte $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ mit Eigenvektoren $v_1, \dots, v_p$)

$$\mathbf{X} = V \Lambda^{\frac{1}{2}} = \left(\sqrt{\lambda_1} v_1, \dots, \sqrt{\lambda_p} v_p \right)$$

3. Multidimensional Scaling & Isomap

Code in `topic07_mds/mds_example_2.py`

Multidimensional Scaling (MDS)

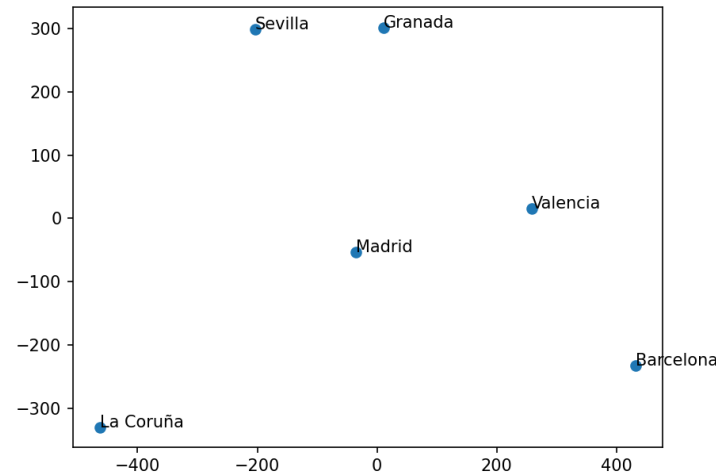
Beispiel

```
cities, distance_matrix = get_distance_matrix()

dist_squared = distance_matrix ** 2
n = len(cities)
H = np.eye(n) - np.ones((n, n)) / n
B = - 1/2 * np.dot(np.dot(H, dist_squared), H)
eigenvalues, eigenvectors = np.linalg.eig(B)

eigenvalues_sqrt = np.sqrt(eigenvalues)
coordinates = np.dot(eigenvectors,
                    np.diag(eigenvalues_sqrt))
```

Ziel Start	Barcelona	Granada	La Coruña	Madrid	Sevilla	Valencia
Barcelona	0	680	900	500	830	300
Granada	680	0	790	360	210	380
La Coruña	900	790	0	510	680	800
Madrid	500	360	510	0	390	300
Sevilla	830	210	680	390	0	540
Valencia	300	380	800	300	540	0

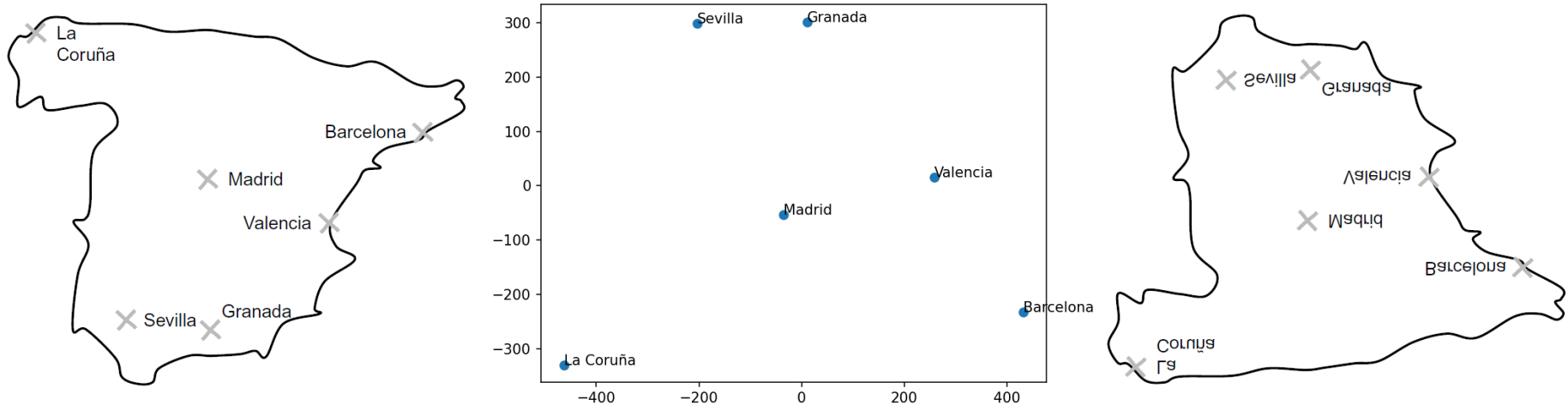


3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

Beispiel

➤ Wie gut ist das Ergebnis?



3. Multidimensional Scaling & Isomap

Code in topic07_mds/
mds_example.py

Multidimensional Scaling (MDS)

Beispiel

- Geht das einfacher?

```
import numpy as np

if __name__ == '__main__':
    cities, distance_matrix = get_distance_matrix()

    dist_squared = distance_matrix ** 2
    n = len(cities)
    H = np.eye(n) - np.ones((n, n)) / n
    B = -1/2 * np.dot(np.dot(H, dist_squared), H)
    eigenvalues, eigenvectors = np.linalg.eig(B)

    eigenvalues_sqrt = np.sqrt(eigenvalues)
    coordinates = np.dot(eigenvectors, np.diag(eigenvalues_sqrt))
```

```
from sklearn.manifold import MDS

if __name__ == '__main__':
    cities, distance_matrix = get_distance_matrix()

    mds = MDS(n_components=2, dissimilarity="precomputed", n_init=100, max_iter=500)
    coordinates = mds.fit_transform(distance_matrix)
```

3. Multidimensional Scaling & Isomap

Code in topic07_mds/
mds_exercise.py

Multidimensional Scaling (MDS)

Übung

Bestimmen Sie mit MDS eine Matrix mit Koordinaten aus der Distanzmatrix.

Hinweis: Benutzen Sie dafür den Code aus topic07_mds/mds_exercise_template.py.

Ziel Start	Berlin	Frankfurt	Hamburg	Köln	München
Berlin	0	548	289	576	586
Frankfurt	548	0	493	195	392
Hamburg	289	493	0	427	776
Köln	576	195	427	0	577
München	586	392	776	577	0

```
DISTANCES = {'BF': 548, 'BH': 289, 'BK': 576, 'BM': 586, 'FH': 493,  
             'FK': 195, 'FM': 392, 'HK': 427, 'HM': 776, 'KM': 577}
```

```
def get_distance_matrix() -> tuple:  
    cities = ['Berlin', 'Frankfurt', 'Hamburg', 'Köln', 'München']  
    distances = np.zeros((5, 5))  
    for idx1, city1 in enumerate(cities):  
        for i, city2 in enumerate(cities[idx1+1:]):  
            idx2 = i + idx1 + 1  
            key = city1[0] + city2[0]  
            distances[idx1, idx2] = DISTANCES[key]  
            distances[idx2, idx1] = DISTANCES[key]  
  
    return cities, distances
```

3. Multidimensional Scaling & Isomap

Multidimensional Scaling (MDS)

- Gegeben: Paarweise Distanzen zwischen n Objekten: $D = (d_{ij})_{i,j=1}^n$
- Gesucht: Datenmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$
- Wie wählen wir die Dimension p der Daten?
- PCA: Anteil der erklärten Varianz (Proportion of Variance Explained)
- MDS: Anteil der erklärten Distanzen (Proportion of Distance Matrix Explained – PDME)

$$\text{PDME}(j) = \frac{\lambda_j}{\sum_{i=1}^n |\lambda_i|}$$

- Wähle p , sodass (kumulierte) erklärte Distanz groß

3. Multidimensional Scaling & Isomap

MDS und PCA

- MDS und PCA sind konzeptionell ähnlich: Berechnung über Eigenwerte und -vektoren
- Wahl der Anzahl der Komponenten/Dimension p über erklärte Varianz/Distanzen
- Wie hängen MDS und PCA genau zusammen?

Wenn

- Datenmatrix $\mathbf{X}$ zentriert ist (d.h. Mittelwertvektor ist 0)
 - Distanzmatrix D aus euklidischen Distanzen besteht
- dann sind MDS-Koordinaten = PCA-Koordinaten

Begründung: vgl. Beginn der Vorlesung (Zusammenhang von $\mathbf{X}^T\mathbf{X}$ (PCA) und $\mathbf{XX}^T$ (MDS))

Warum benötigen wir überhaupt MDS?

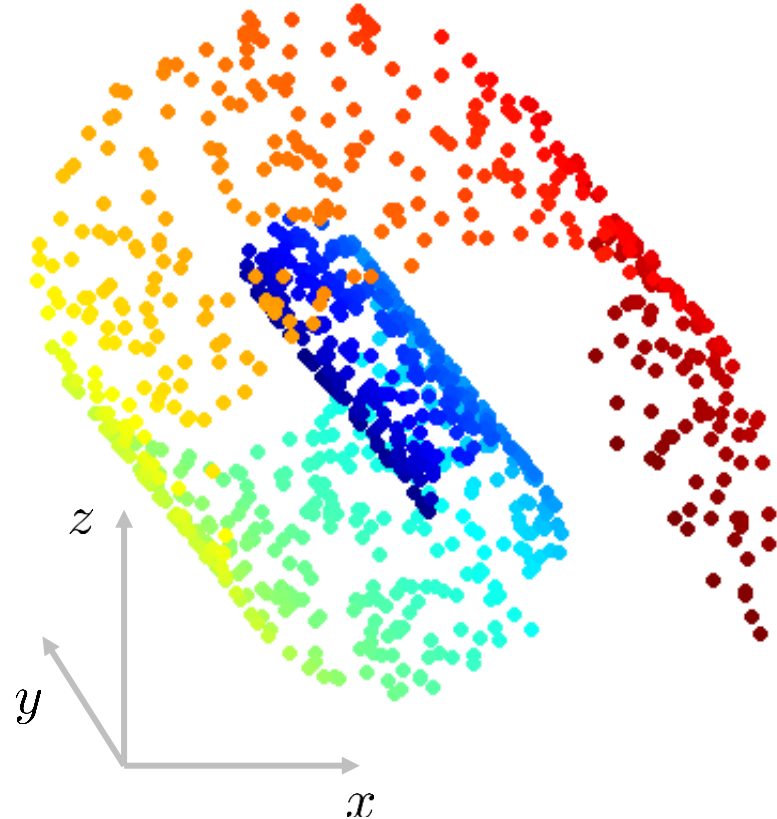
- Andere Ausgangslage (Distanzen bekannt, Koordinaten unbekannt)

3. Multidimensional Scaling & Isomap

Grenzen der PCA

Arbeiten Sie zu zweit.

- Welche Hauptkomponenten erwarten Sie hier?
 - Was macht den dargestellten Datensatz für die PCA so problematisch?
-
- Daten liegen auf einer nicht-linearen Mannigfaltigkeit
 - PCA hilft uns lineare Strukturen zu entdecken → keine Hilfe bei nicht-linearen Strukturen

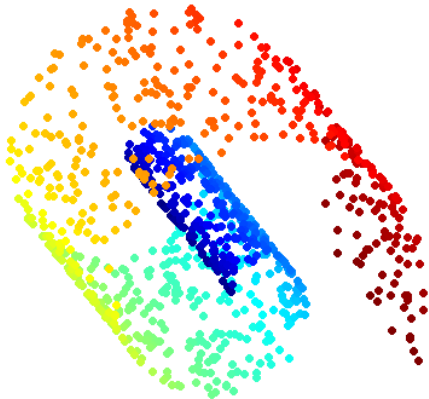


3. Multidimensional Scaling & Isomap

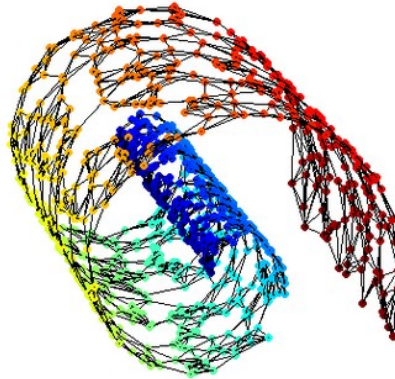
Grenzen der PCA

- Alternativ: MDS
- Besser gesagt: Isomap (isometric feature mapping)

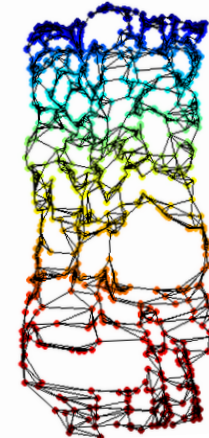
Welche Parameter werden Sie bei Verwendung von *isomap* wählen müssen?



Erzeugung eines Nachbarschaftsgraphen: Datenpunkte, die nah sind, werden mit einer Kante verbunden



Distanz zwischen zwei Punkten: Länge des kürzesten Pfades



MDS: Distanzmatrix liefert Eigenvektoren zur Dimensionsreduktion

3. Multidimensional Scaling & Isomap

Isomap

- Gegeben: Datenmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$
- Gesucht: Niedrigdimensionale Projektion $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times d}$ mit $d < p$

Algorithmus:

1. Ermittle Nachbarschaftsgraph
2. Ermittle paarweise Distanzen zwischen Punkten auf dem Graph
3. Wende MDS auf Matrix D der paarweisen Distanzen an

Multivariate EDA

Rückblick

- Wie können wir für hochdimensionale Daten ($p > n$) effizient die Hauptkomponenten bestimmen?
- Was ist beim Multidimensional Scaling gegeben und gesucht?
- Wie kann mit MDS die Dimension reduziert werden?
- Aus welchen 3 Schritten besteht der MDS-Algorithmus?
- Wie wählen wir die Dimension der Datenmatrix $X \in \mathbb{R}^{n \times p}$
- Wie hängen PCA und MDS zusammen?
- Wie funktioniert der Isomap-Algorithmus?
- Was ist der Unterschied zwischen Dimensionsreduktion mit MDS und Isomap?
- Würden Sie für Daten in einem linearen Raum PCA oder Isomap zur Dimensionsreduktion wählen?
- Würden Sie für Daten in einem nicht-linearen Raum PCA oder Isomap zur Dimensionsreduktion wählen?